

Beginners Guide

notes taken from [Andrew Bellini's DEF CON 32 talk](#)

If you get a shell on an embedded system

- EnumerateEnumerateEnumerateEnumerateEnumerate
- Review /etc and /var folders
- Look for passwords, hardcoded keys, endpoints(addresses that the devices calls home to)
- What processes and network connections are up? (ps aux & netstat)

Acquiring Firmware

- Embedded Linux has:
 - Bootloader
 - Kernel
 - Root file system
 - Partitions for data
 - there's likely a partition for writable data. this could store loot (on a router) such as:
 - passwords set by user
 - pre-shared key
- How to acquire firmware
 - Download from vendor
 - Trivial File Transfer Protocol (TFTP)
 - Hacker can host a TFTP server and download files, or even the entire root system, from the embedded device
 - Dump firmware from device

Reading SPI flash chips (timestamp: 32:07)

- Most chips use an 8 pin SPI flash chip
 - Use the flash reader to read in-circuit
- The hacker can use [flashrom](#) (Free/open source)
 - With a ch341 programmer, the hacker can run
 - `flashrom -p ch341a_spi -r name.bin`

Reverse Engineering

Unpacking firmware

1. Hope that it's not encrypted :)
2. Use binwalk to check contents and extract firmware
 - `binwalk -Me firmware_name.bin`

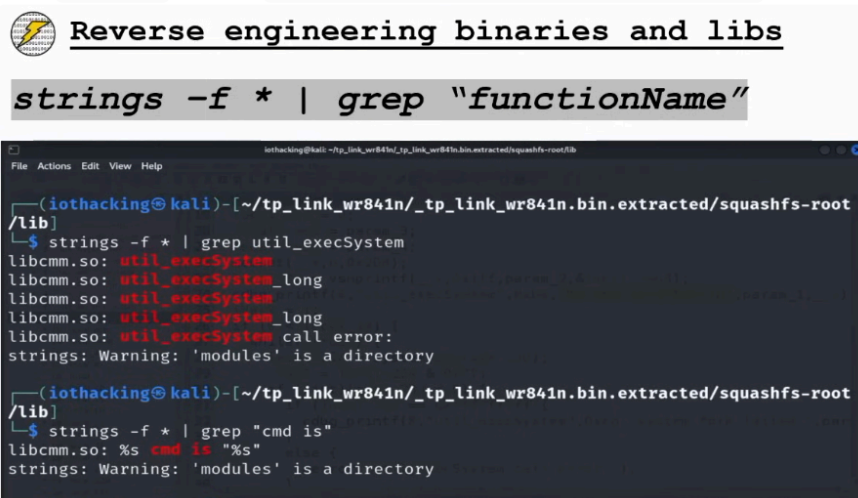
Analyzing Firmware

- Focus on root file system
 - `find . -name "*.extension" (.xml, .txt, .json)`
- Enumerate files of interest
 - `strings -f * | grep "pass"` (also try replacing "pass" with `api`, `key`, `.com` and other TLDs (to look for endpoints))
- Read rcS scripts
 - Most IoT devices use busybox to initialize systems. busybox combines all binaries into a singular binary before using that output binary as the initialization system.
 - Busybox runs all rcS scripts in `/etc/init.d`

Reverse Engineering Binaries & Libraries

This is where the hacker is most likely to find a vulnerability.

- Go with a "forward down" approach. In other words, start with custom binaries and libraries that are unique to the device.
- Interact with the device as it was designed to be used and follow device logging to gain deeper insights into how the device operates.
 - Watch Andrew go through this with an example at timestamp: (41:29)
- Once a library is chosen as a target, go back and enumerate specifically on that library
 - Run strings again: `strings -f * | grep "functionName"`



The screenshot shows a terminal window with the following content:

```
(iothacking@kali) - [~/tp_link_wr841n/_tp_link_wr841n.bin.extracted/squashfs-root]
[lib]
$ strings -f * | grep util_execSystem
libcmm.so: util_execSystem
libcmm.so: util_execSystem_long
libcmm.so: util_execSystem
libcmm.so: util_execSystem_long
libcmm.so: util_execSystem call error:
strings: Warning: 'modules' is a directory

(iothacking@kali) - [~/tp_link_wr841n/_tp_link_wr841n.bin.extracted/squashfs-root]
[lib]
$ strings -f * | grep "cmd is"
libcmm.so: %s cmd is "%s"
strings: Warning: 'modules' is a directory
```

Core Workflow

- Interact with the device and trace logging to underlying binaries and libraries
 - Check for user inputs that are sent as system calls
 - If a user's input gets sent to a system call, the hacker must look for a command injection vulnerability
 - Check for user inputs sent to unsafe C functions
 - E.g. `memcpy`, `strcpy`, `strcat`, `printf`, `sprint`, `gets`, `scanf`, `fscanf`

- Since these functions allocate a specific amount of memory, buffer overflows are possible.